ENM 5310: Data-driven Modeling and Probabilistic Scientific Computing

Lecture #18: Transformers in pictures, and deep learning highlights



Today

1

The transformer, visually

Attention → self-attention → positions → heads → the block → the three families

≈ 35 min

2

Deep learning highlights

Scaling, foundation models, vision–language and multimodal models, AlphaFold

≈ 35 min

3

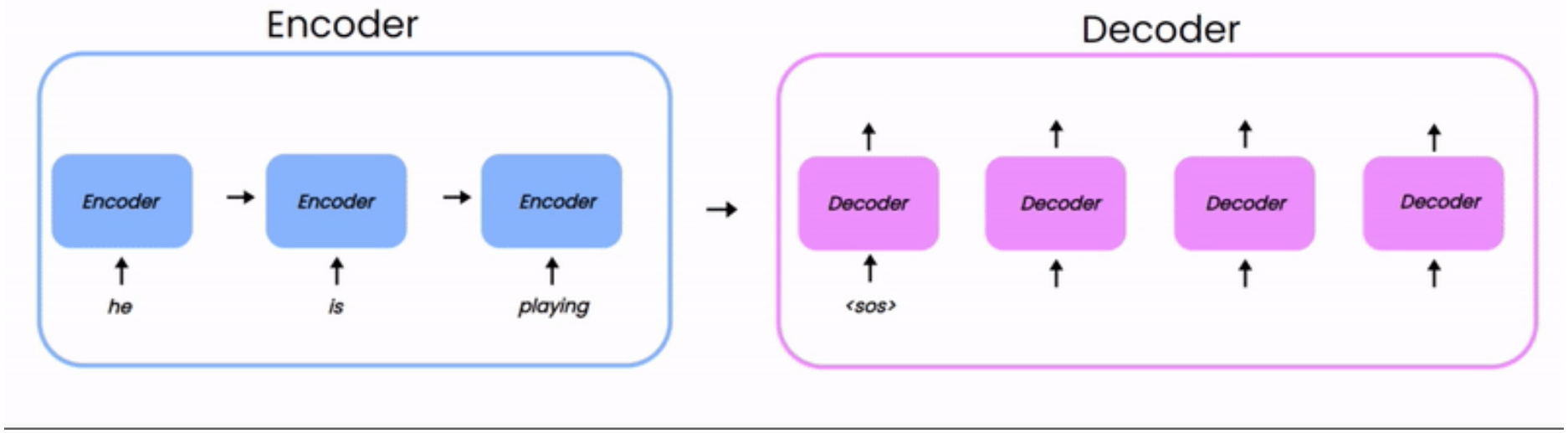
Physics & engineering

Transformers for science, PINNs, neural operators — setting up next lecture

≈ 15 min

Last time: the transformer on the board. Today: the same ideas in pictures — then what they have made possible.

Where we started: sequence-to-sequence with RNNs



1 Sequential

h_t needs h_{t-1}: no parallelism over time, on hardware built for parallel matrix products

2 Long paths

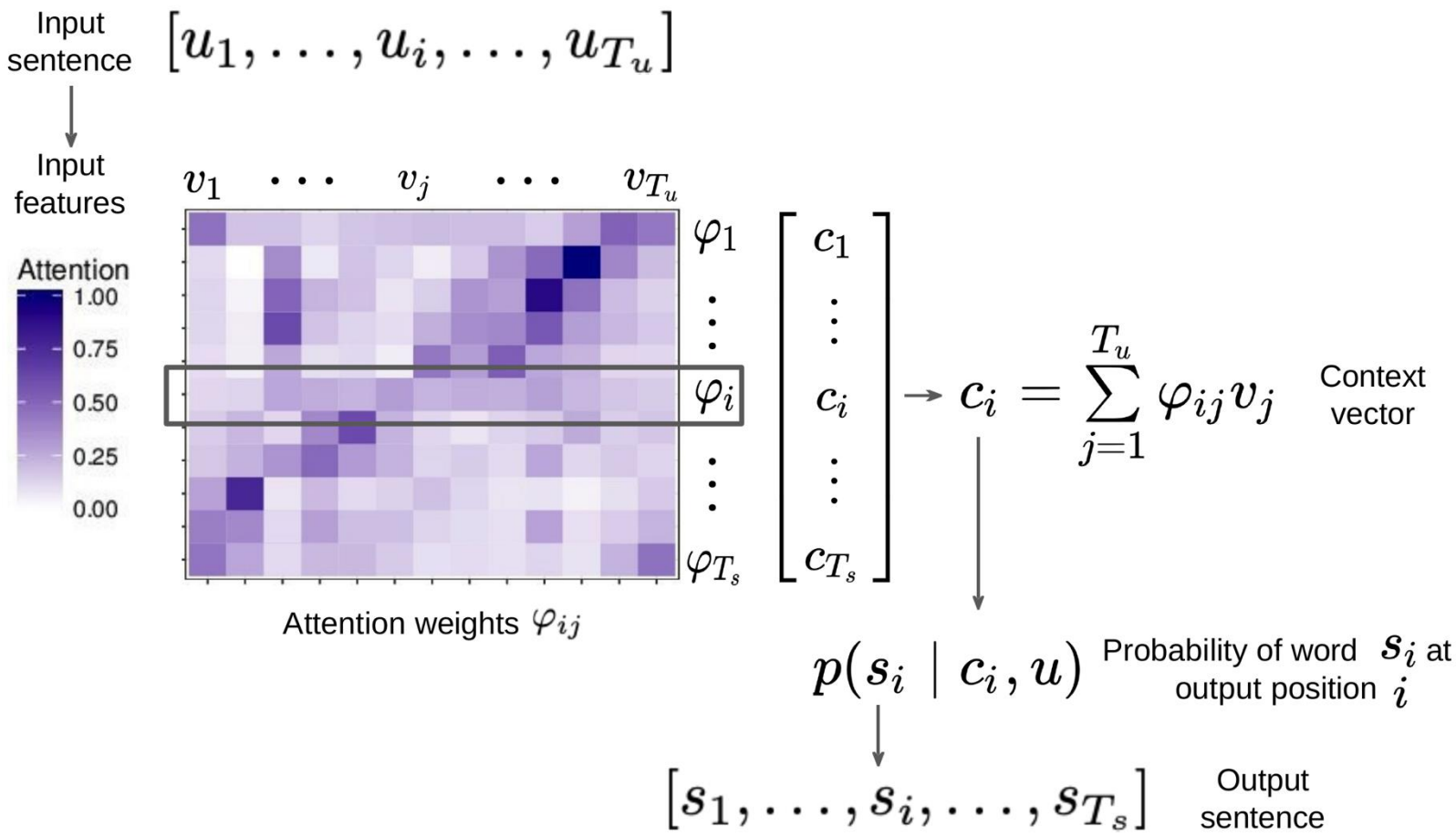
x_1 reaches the decoder only through S recurrent steps: a product of Jacobians, so gradients vanish

3 Bottleneck

The whole input is squeezed into one fixed-size vector $m = h_S$, however long the input

Attention was invented to fix the bottleneck — and ended up fixing all three.

Bahdanau attention: look back at every input



Score every encoder state against the current decoder state

Softmax → weights ϕ_{ij}: a learned **soft alignment**

Context $c_i = \Sigma_j \phi_{ij} v_j$ is recomputed at every output step

Nobody labels the alignments. They emerge because they reduce the translation loss — and the bottleneck is gone.

Bahdanau, Cho & Bengio (2015). Neural machine translation by jointly learning to align and translate. ICLR.

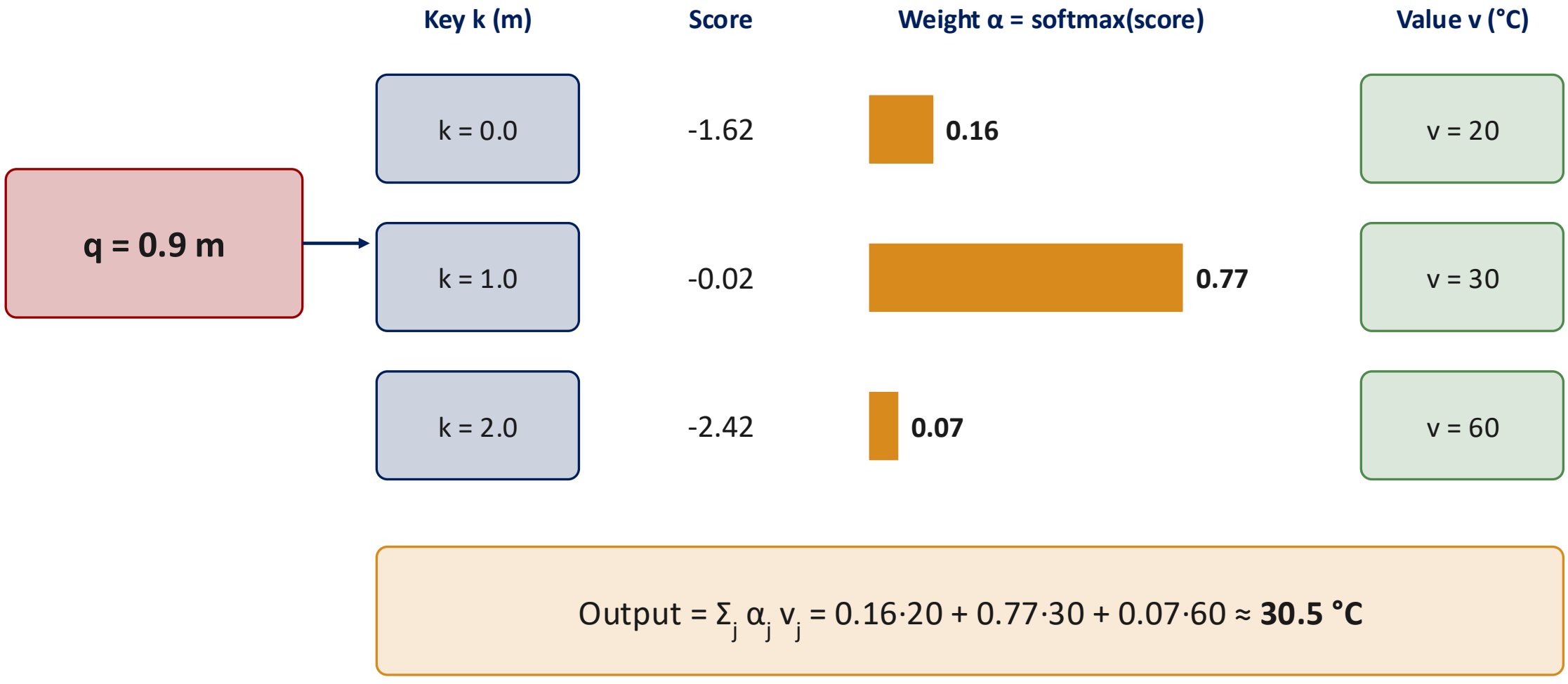
Attention is a soft dictionary lookup

Query: what I am looking for

Keys: what each item advertises (for matching)

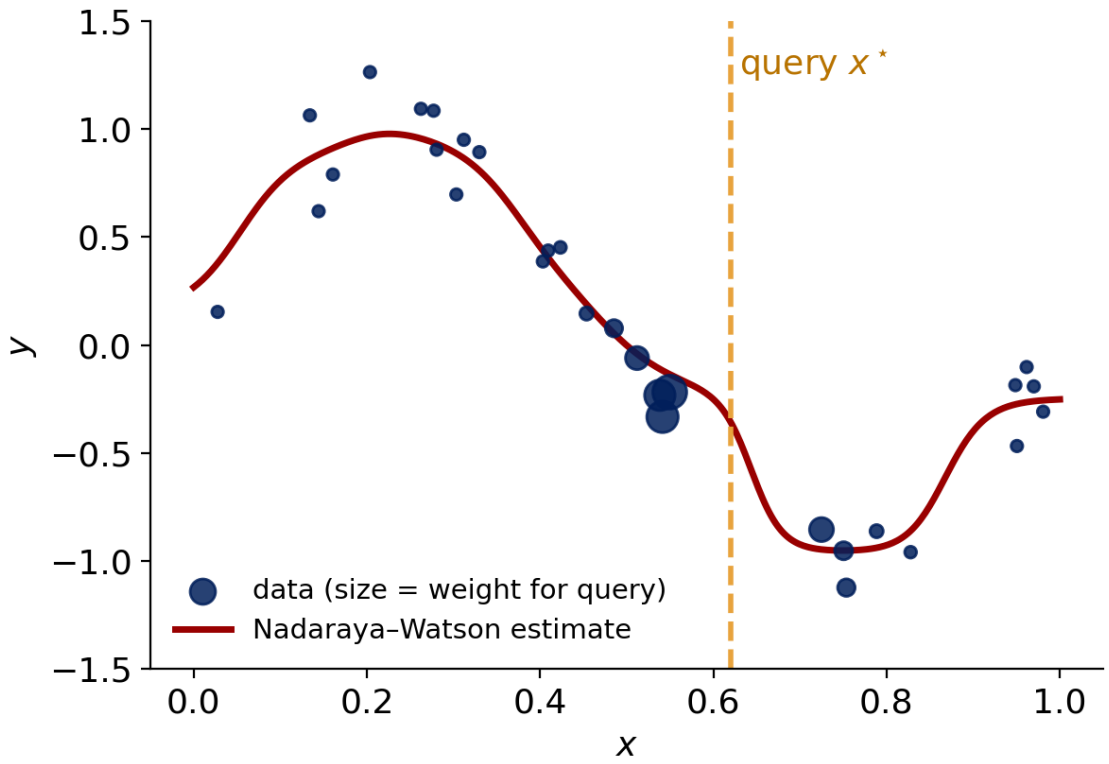
Values: what each item returns

Example: estimate the temperature at q = 0.9 m on a beam from three sensors, score = −(q − k)² / 2ℓ², ℓ = 0.5 m



Small ℓ (low temperature) → nearly one-hot: a hard lookup. Large ℓ → a plain average. Unlike a hard lookup, the soft one is differentiable — so queries, keys and values can be learned.

You have seen this before: kernel regression



Nadaraya–Watson with a Gaussian kernel:

$$\hat{f}(x^\star) = \Sigma_j \, \mathrm{softmax}_j\left(-\|x^\star - x_j\|^2 / 2\ell^2 \right) y_j$$

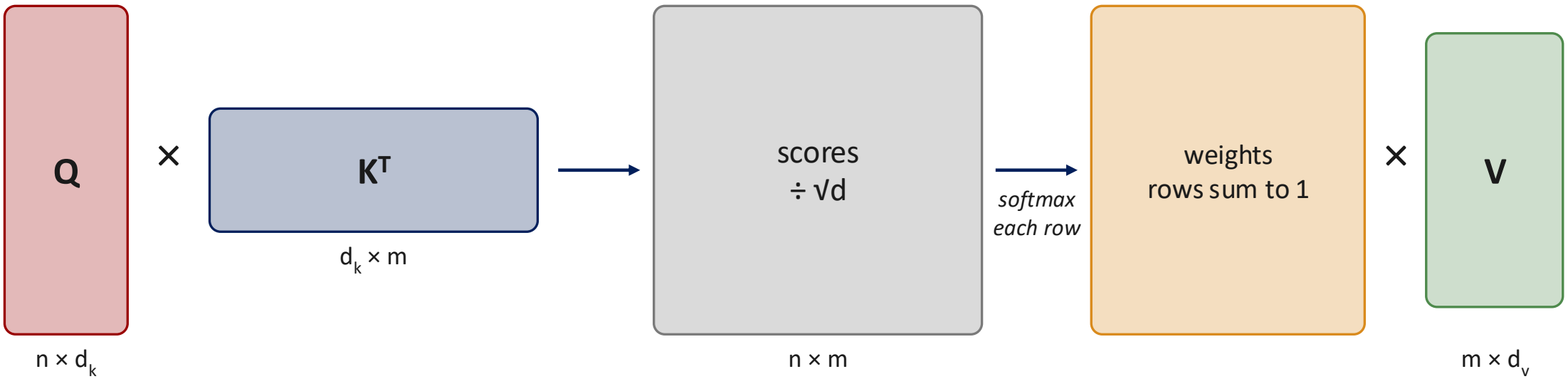
- query = $x^\star$, keys = x_j, values = y_j
- Expand the score: $-\|q\|^2/2\ell^2 + q^\top k_j/\ell^2 - \|k_j\|^2/2\ell^2$
- First term: same for every j → drops out of the softmax
- Last term: drops out if all keys have equal norm

What remains is **dot-product attention**: $\alpha_j = \mathrm{softmax}_j(q^\top k_j / \ell^2)$. The bandwidth ℓ^2 is a temperature.

Compare the GP mean (Lecture 9): $\mu(x^\star) = k(x^\star, X)(K + \sigma^2 I)^{-1} y$ — also a weighted sum of targets, but its weights account for correlations between training points. What attention adds is **learning**: queries and keys come from learned projections.

Scaled dot-product attention: all lookups at once

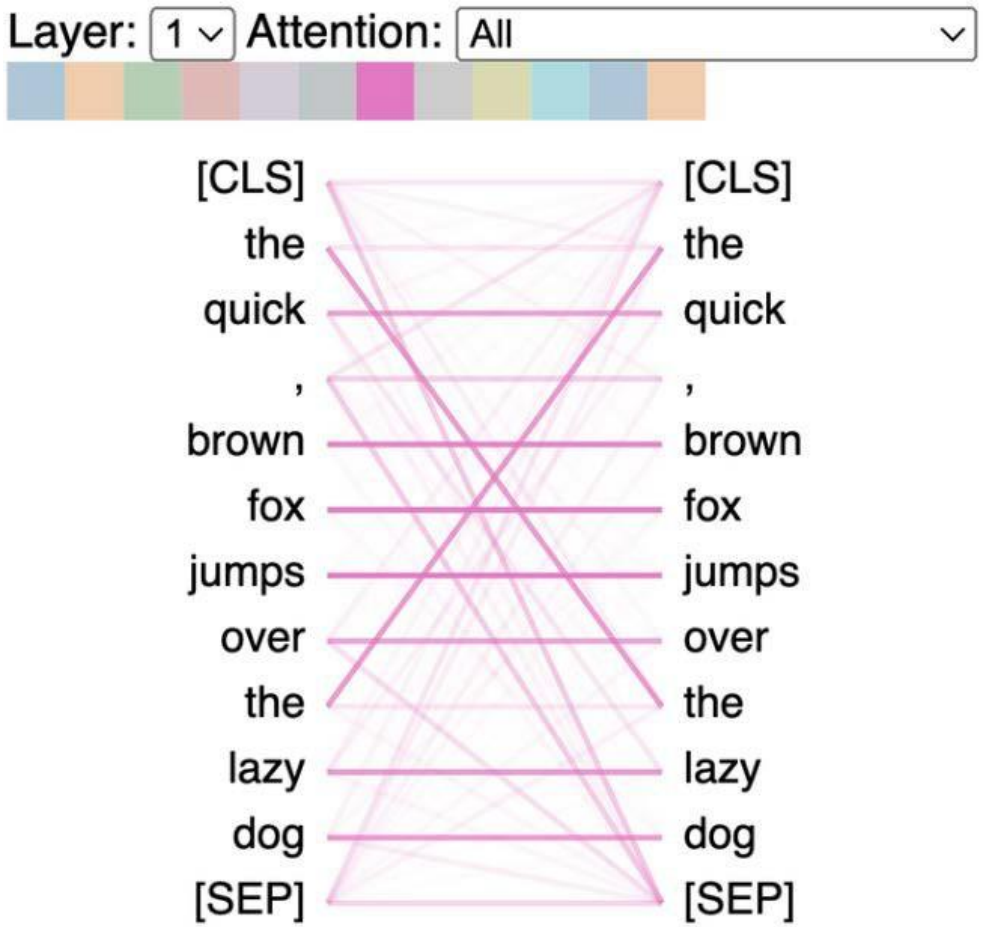
$$\textbf{Attention}(Q, K, V) = \textbf{softmax}\left(Q K^T / \sqrt{d_k} \right) V$$



→ *output: $n \times d_v$ (row i = query i's soft lookup)*

Why divide by $\sqrt{d_k}$? If the entries of q and k have unit variance, $\mathrm{Var}[q^T k] = d_k$. With $d_k = 64$ the scores have std 8, the softmax is nearly one-hot, and its gradient is nearly zero — the saturation problem of Lecture 13. Dividing by $\sqrt{d_k}$ restores unit variance.

Self-attention: every token reads from every other



Attention weights of one head in a trained language model (BertViz)

- Queries, keys and values are three projections of the **same** tokens: $Q = XW_Q$, $K = XW_K$, $V = XW_V$
- Every output row is a content-weighted average of **all** value rows
- **Global**: token 1 reads token 1000 in one step
- **Parallel**: all rows at once, by matrix products — no recurrence

"The animal didn't cross the street because **it** was too tired." The query of *it* looks for a referent; the key of *animal* advertises one. Trained heads learn exactly this.

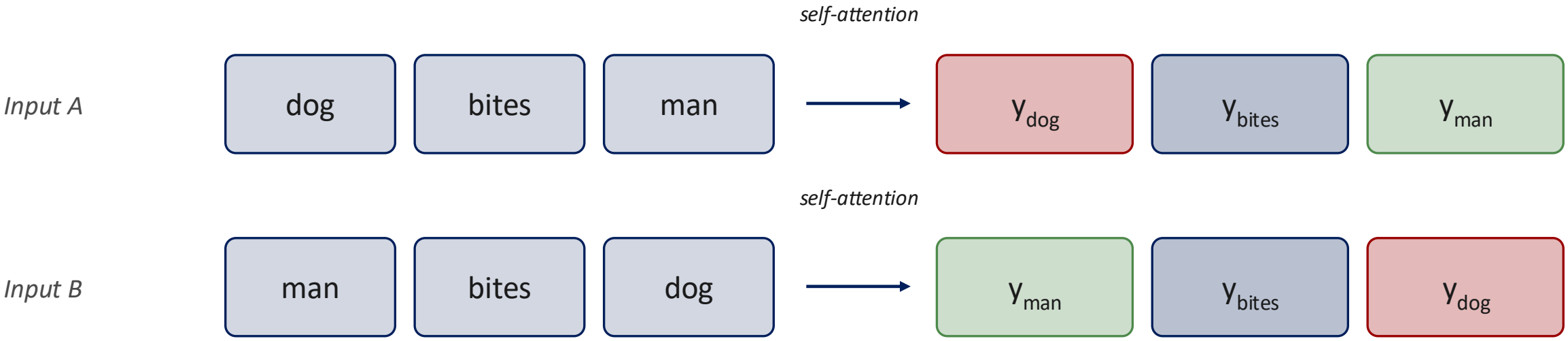
Vig (2019). A multiscale visualization of attention in the transformer model. ACL (demos).

How layers mix information across positions

Layer	Output at position i	Mixing weights are…	Reach	Parallel?
Dense (MLP)	$\Sigma_j W_{ij} x_j$	fixed after training, one per pair	global	yes
Convolution	$\Sigma_j w_{j-i} x_j$	fixed, shared, depend only on offset	local	yes
Recurrence	$h_i = f(h_{i-1}, x_i)$	implicit, passed through a state	long paths	no
Self-attention	$\Sigma_j \alpha_{ij}(X) v_j$	**computed from the content**	global	yes

A convolution applies the same stencil everywhere, whatever the data. Self-attention recomputes its “stencil” for every input: which positions matter, and how much, depends on what is in them.

Self-attention sees a set, not a sequence



Same vectors, shuffled: f(PX) = P f(X) for any permutation P

Right for sets

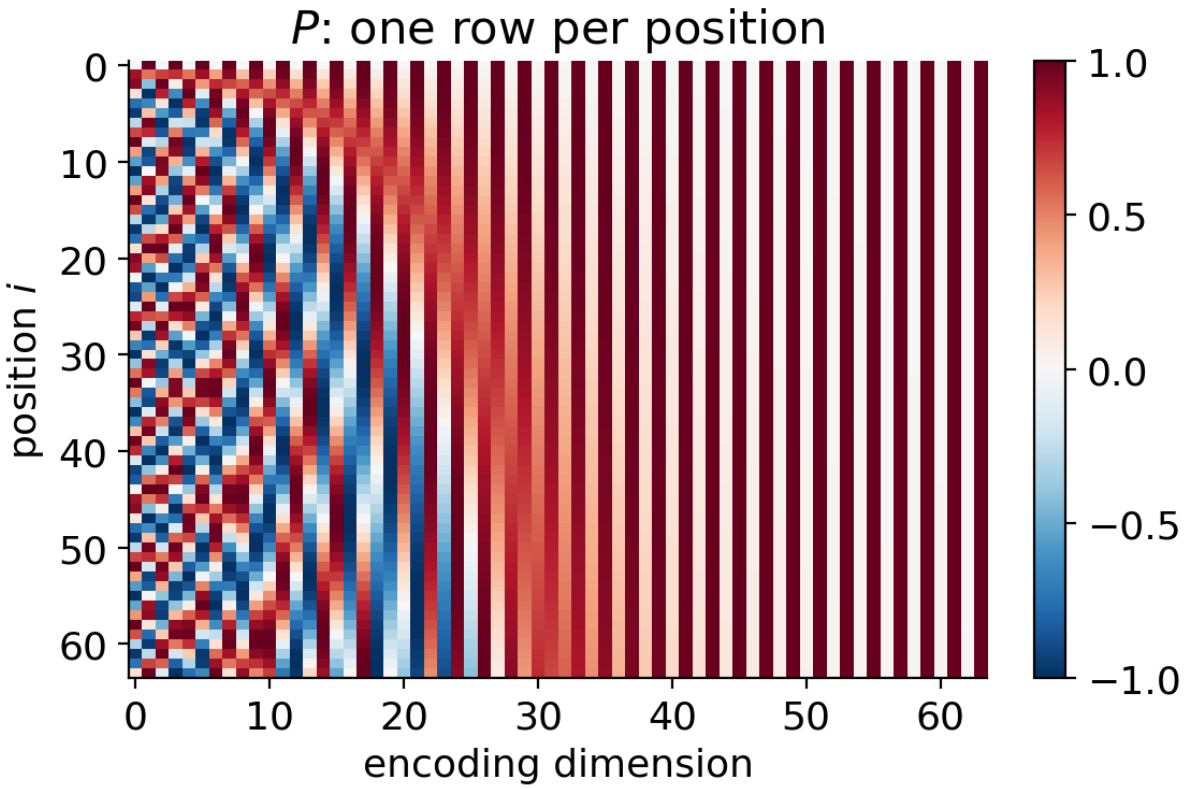
Particles, atoms, sensors, mesh nodes: the order in which we list them is arbitrary, and the model should not care.

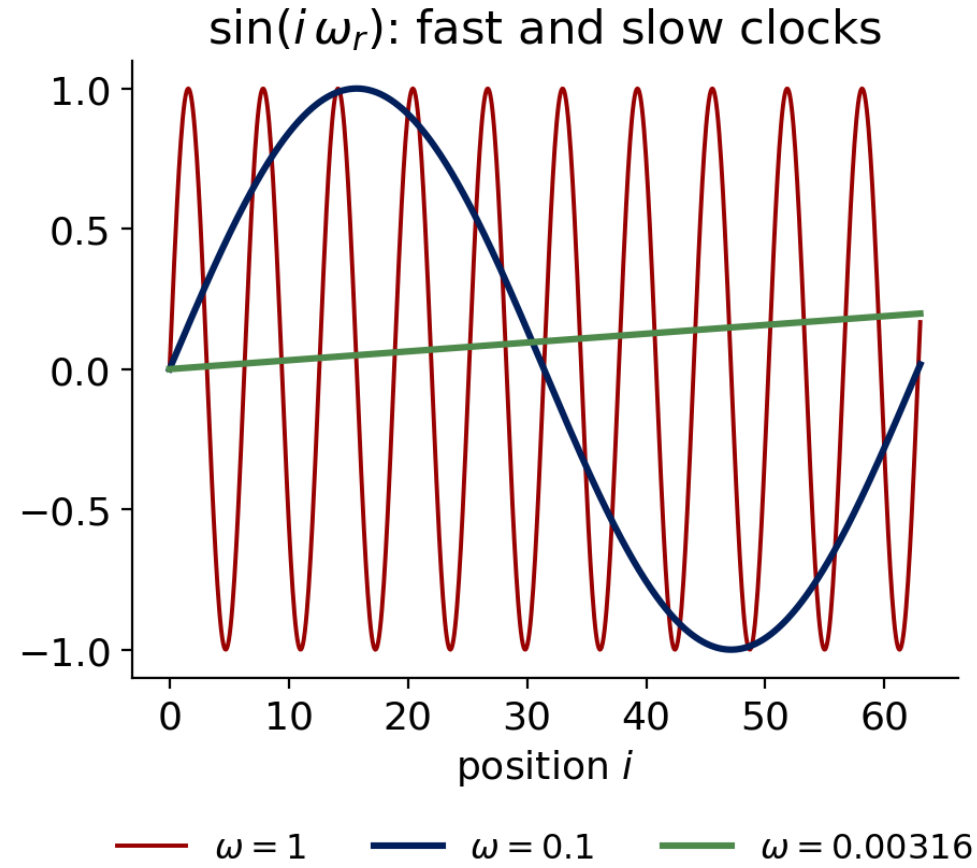
Wrong for sequences

“dog bites man” ≠ “man bites dog”. Text, time series and ordered grids need position — so we must add it.

Compare convolution (Lecture 14): equivariant to translations only — it keeps relative position for free.

Positional encodings: multi-frequency clocks





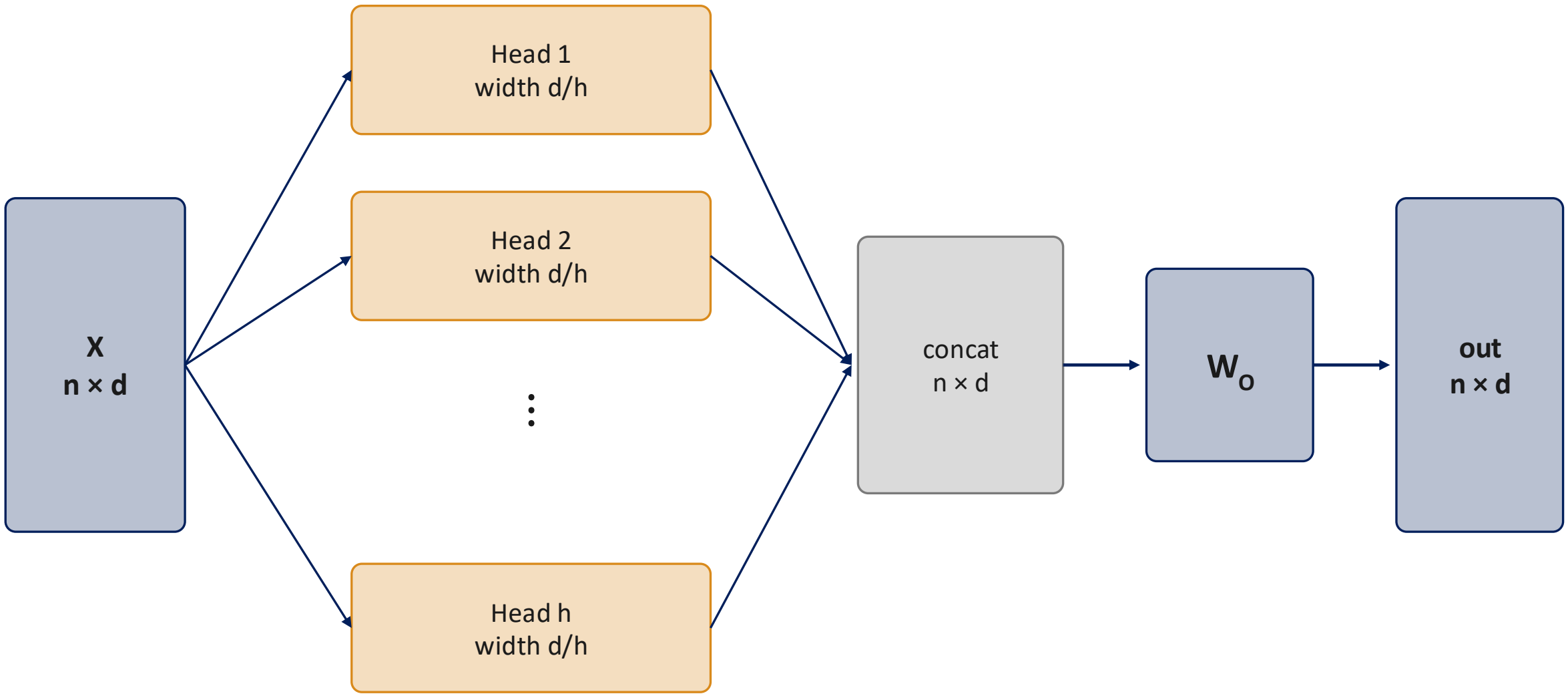
$$x_i \leftarrow x_i + p_i, \quad p_{i,2r} = \sin(i\,\omega_r), \quad p_{i,2r+1} = \cos(i\,\omega_r), \quad \omega_r = 10000^{-2r/d}$$

Fast clocks tell neighbours apart; slow clocks tell distant positions apart — smooth “digits” of the position.

A shift by Δ is a rotation of each (sin, cos) pair, so $p_i^T p_j = \Sigma_r \cos((i-j)\omega_r)$ depends only on $i - j$.

For physical tokens, use Fourier features of the coordinates (x, y, z, t) — the same idea.

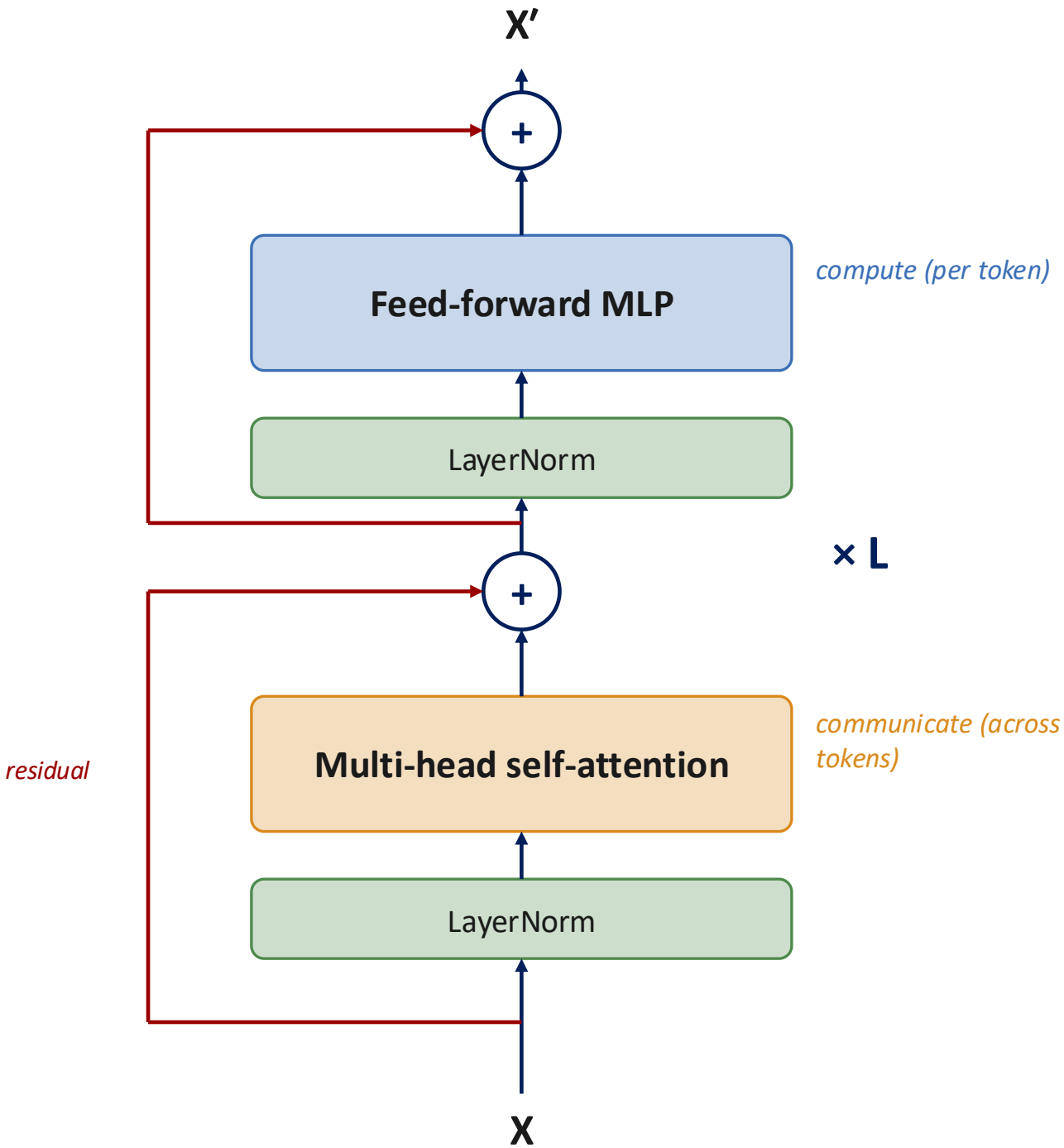
Multi-head attention: several lookups in parallel



Why: one softmax cannot look sharply at several unrelated places. Different heads learn different relations — a referent, a verb, the next word; for particles, short- and long-range interactions.

Cost: h heads of width d/h use $3d^2 + d^2 = 4d^2$ parameters — the same as one full-width head. In code, the heads are a reshape.

The transformer block: communicate, then compute



Z = X + MHA(LN(X))

X' = Z + FFN(LN(Z))

FFN(z) = GELU($zW_1 + b_1$) $W_2 + b_2$, d → 4d → d, applied to each token separately

A meeting: in the attention sublayer every token listens to everyone and takes notes, weighted by relevance; in the FFN each token goes off and thinks alone about what it heard. A transformer is L rounds of "talk, then think."

Pre-LN vs post-LN: modern models normalize before each sublayer, keeping a clean identity path for gradients. The 2017 paper normalized after the addition ("Add & Norm"), which needs careful learning-rate warmup.

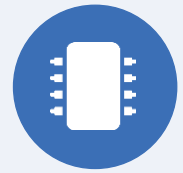
What each piece of the block is for



Multi-head attention

Mixes information across tokens, with content-dependent weights.

Without it: tokens never interact — an MLP on each token.



Feed-forward MLP

Transforms each token nonlinearly; holds about two-thirds of the parameters.

Without it: outputs are only averages of values; deep attention-only stacks collapse in rank.



Residual connections

Each sublayer adds a correction to a running “residual stream”.

Without it: gradients pass through every layer’s Jacobian — depth becomes hard (Lecture 13).



LayerNorm

Re-standardizes each token before each sublayer.

Without it: scales drift with depth; softmax and GELU saturate.

Per block: $4d^2$ (attention) + $8d^2$ (FFN) = **$12d^2$** parameters. Cost: $O(n\,d^2 + n^2\,d)$.

The original transformer (Vaswani et al., 2017)

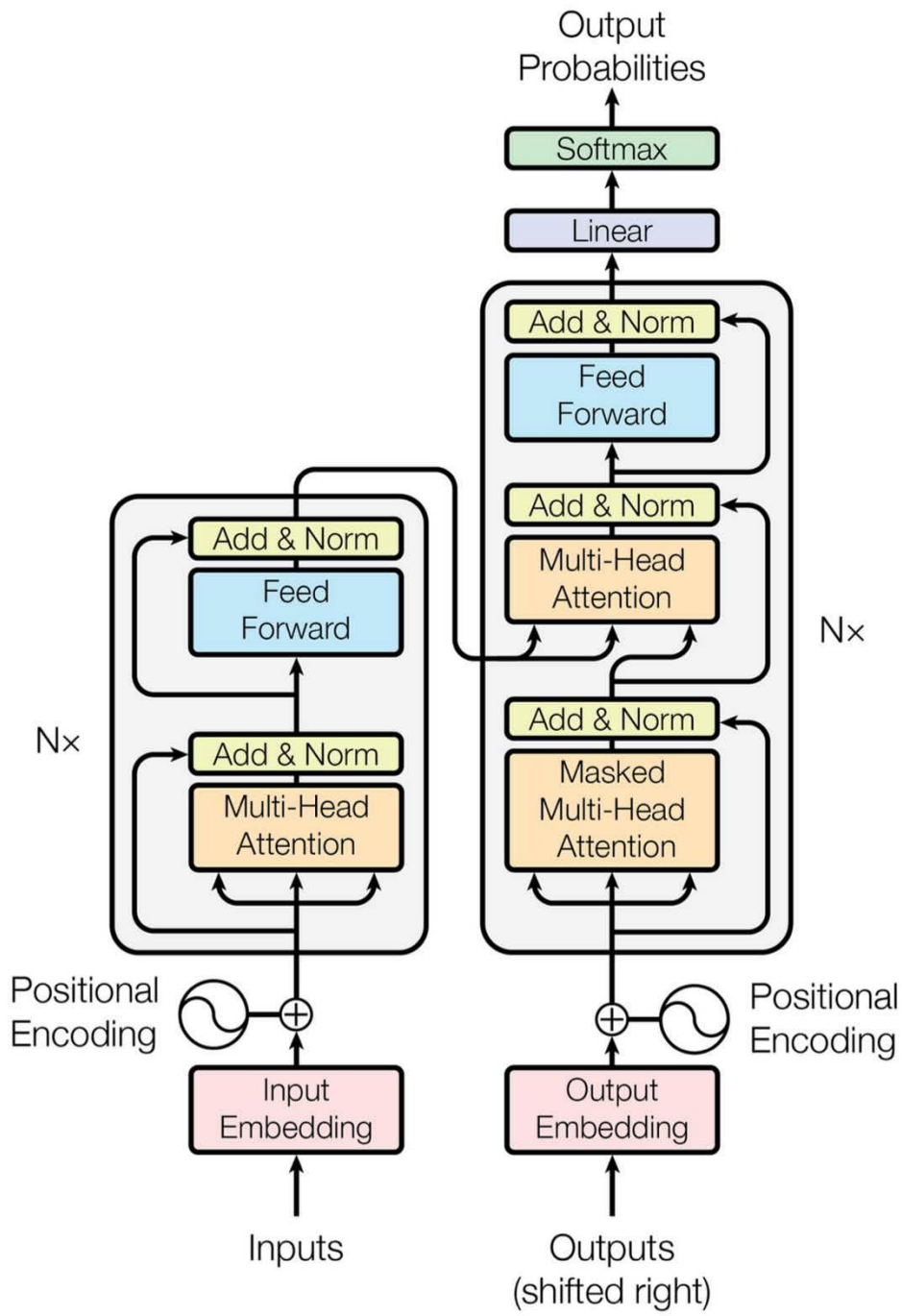


Figure 1: The Transformer - model architecture.

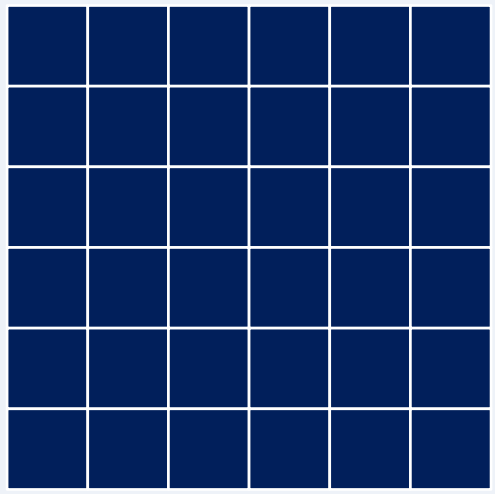
- **Left stack — encoder:** no mask, every input token sees every other
- **Right stack — decoder:** masked (causal) self-attention, then **cross-attention** to the encoder output — Bahdanau attention, reborn
- **Add & Norm** after each sublayer = post-LN; modern models put LN first
- **Base model:** N = 6 blocks per stack, d = 512, h = 8, d_{ff} = 2048 — about 65 M parameters
- Trained for translation in about 12 hours on 8 GPUs

Vaswani et al. (2017). Attention is all you need. NeurIPS.

One block, three masks: the transformer families

Encoder-only

no mask — sees everything



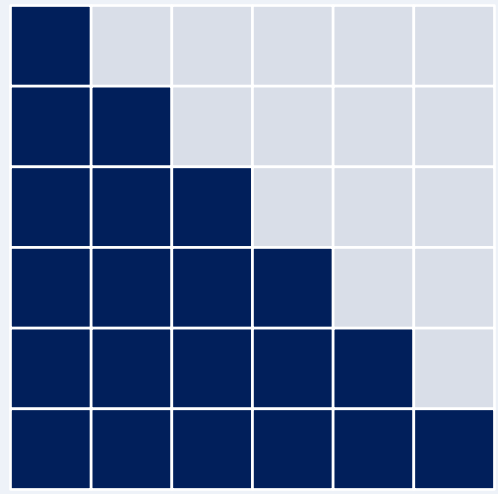
query i ↓ key j →

e.g. **BERT, ViT**

Understand a whole input: classify, regress, label every token

Decoder-only

causal mask — sees the past



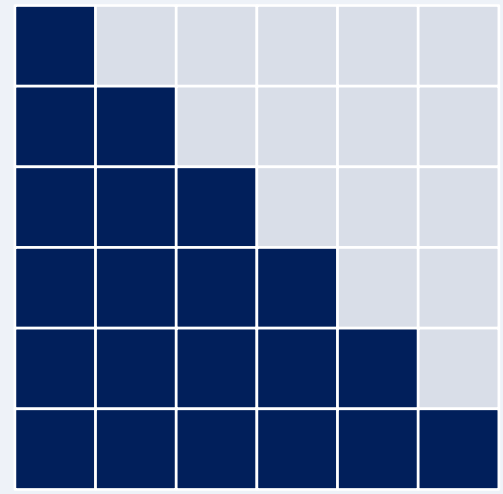
query i ↓ key j →

e.g. **GPT and most LLMs**

Generate sequences one token at a time

Encoder–decoder

causal decoder + cross-attention



query i ↓ key j →

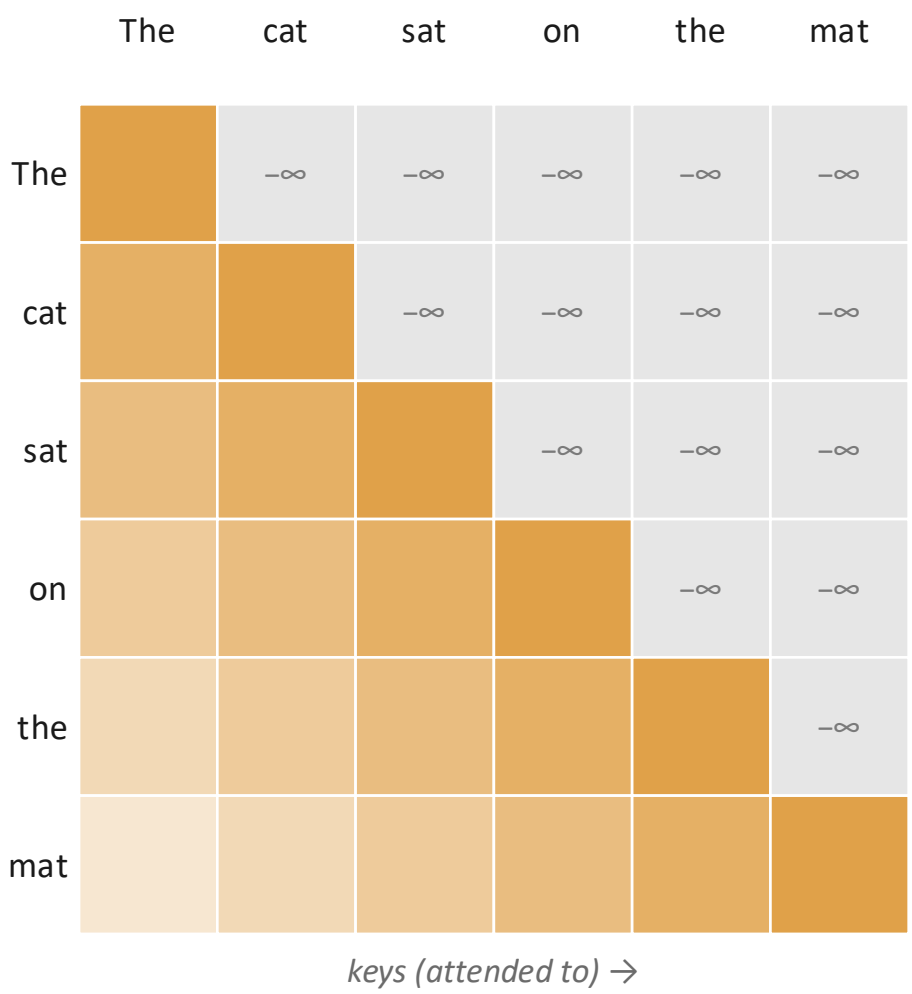
+ cross-attention: queries from the decoder, keys and values from the encoder

e.g. **Original transformer, T5**

Map one sequence to another: translation, forcing → response

Devlin et al. (2019) BERT · Radford et al. (2018) GPT · Raffel et al. (2020) T5 · Dosovitskiy et al. (2021) ViT

The causal mask: parallel training, sequential generation



Scores + M, with $M_{ij} = -\infty$ for $j > i$: future positions get weight $e^{-\infty} = 0$

$p(y_{1:n}) = \Pi_t\, p(y_t \mid y_{<t})$ — Lecture 15

Training: one pass

Feed the whole true sequence. The mask guarantees row t only sees y≤t, so one forward pass gives all n next-token predictions — teacher forcing for the entire sequence at once. An RNN needed n sequential steps.

Generation: one token at a time

Sample the next token, append it, run again. Past keys and values never change, so they are cached (the “KV cache”) and each step only processes the newest token.

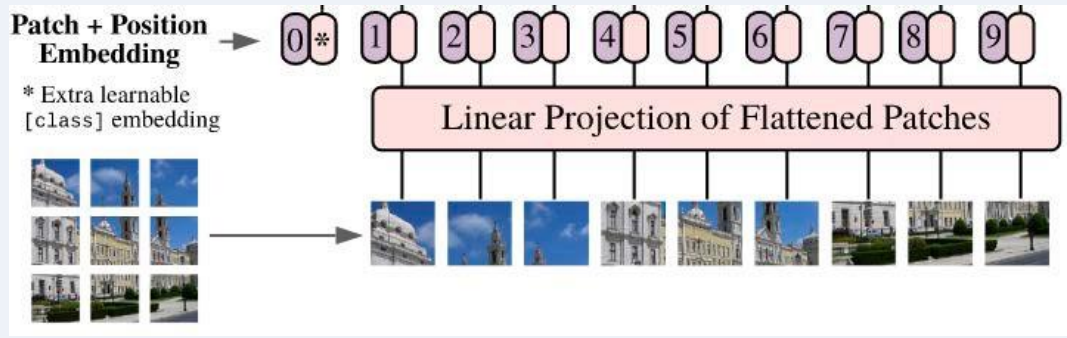
Anything can be a token

Text: subwords → ids → embedding rows

Many	words	map	to	one	token
8607	4339	2472	311	832	4037

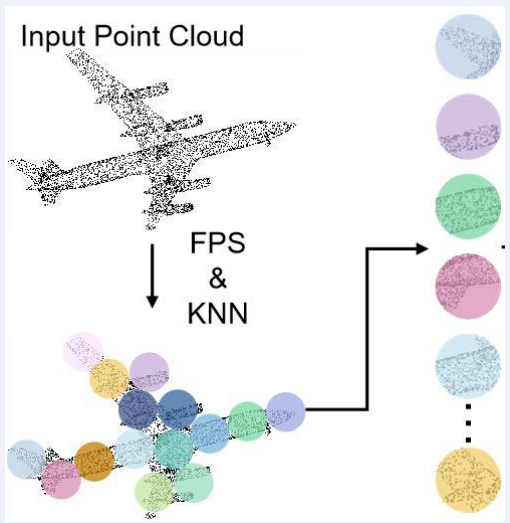
Each id selects a row of a learned embedding matrix $E \in \mathbb{R}^{|V| \times d}$; tens of thousands of subword tokens.

Images: 16×16 patches (ViT)



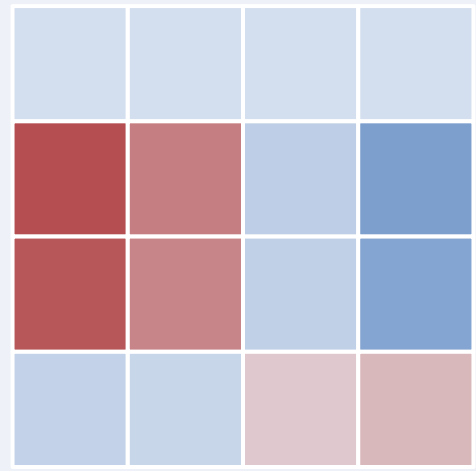
224 × 224 image → 196 patch tokens, each mapped to d dimensions by one linear layer.

Point clouds: local groups



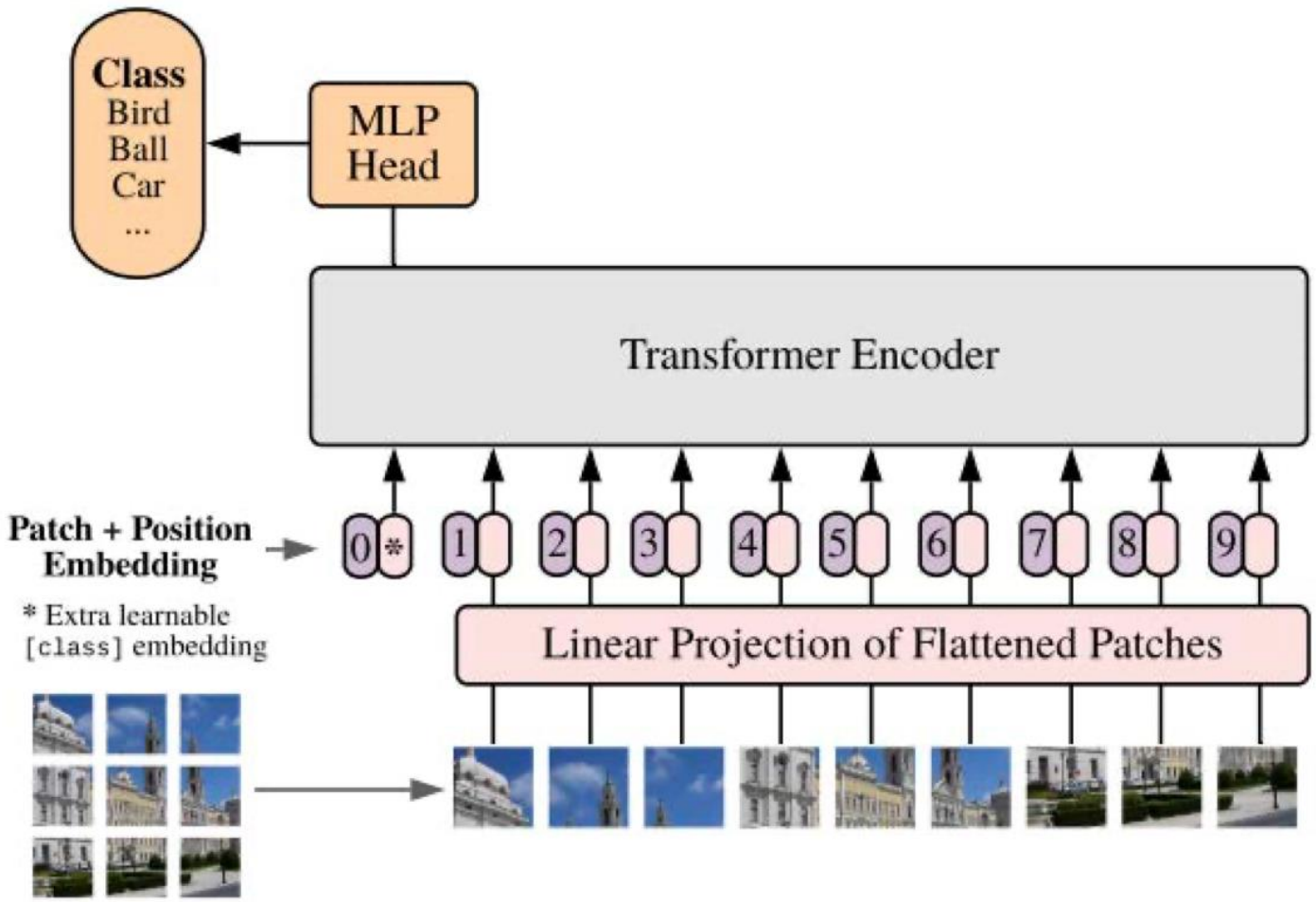
Sample centres (farthest-point sampling), group nearest neighbours, embed each group as one token.

Physical fields and sensors



256×256 field in 16×16 patches → 256 tokens (not 65,536). Irregular points: one token each, with Fourier features of the coordinates as positions.

Vision Transformer: an image is worth 16×16 words



- Cut the image into patches, embed each linearly, add positions
- An **encoder-only** stack; classify from an extra [class] token (or the average of all tokens)
- Inductive bias: none for locality or translation — it must learn them
- Trails comparable CNNs on ImageNet alone; overtakes them with large-scale pretraining

Little data: structure built into the architecture wins (CNNs, GPs). Lots of data: a flexible model wins.

Dosovitskiy et al. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. ICLR.

Under the hood: a GPT-style model in ~20 lines

```
def attention(q, k, v, mask):                    # q, k, v: (h, n, dk)
    s = q @ k.swapaxes(-1, -2) / np.sqrt(q.shape[-1]) + mask
    return softmax(s) @ v                        # row-wise softmax

def mha(x, p, h, mask):                          # x: (n, d)
    n, d = x.shape
    split = lambda t: t.reshape(n, h, d // h).transpose(1, 0, 2)
    y = attention(split(x @ p.Wq), split(x @ p.Wk), split(x @ p.Wv), mask)
    return y.transpose(1, 0, 2).reshape(n, d) @ p.Wo   # concat heads, mix

def block(x, p, h, mask):                        # pre-LN transformer block
    x = x + mha(layer_norm(x), p, h, mask) # communicate
    x = x + ffn(layer_norm(x), p)          # compute: d -> 4d -> d
    return x

def gpt(tokens, P):
    n = len(tokens)
    x = P.E[tokens] + positional_encoding(n, P.d)
    mask = np.triu(np.full((n, n), -np.inf), k=1)    # causal
    for p in P.blocks:
        x = block(x, p, P.h, mask)
    return layer_norm(x) @ P.Wu              # next-token logits (n, |V|)
```

Heads are a reshape. Encoder vs decoder is one mask. The full version (LayerNorm, FFN, positional encoding, loss) is in the Lecture 17 notes, §7.

Counting: parameters and cost

$12\,d^2$

parameters per block

$4d^2$ attention (W_Q, W_K, W_V, W_O) + $8d^2$ FFN ($d \to 4d \to d$)

175 B

GPT-3 sanity check

$12 \cdot 96$ blocks $\cdot\ 12{,}288^2 \approx 1.74 \times 10^{11}$

$O(n\,d^2 + n^2\,d)$

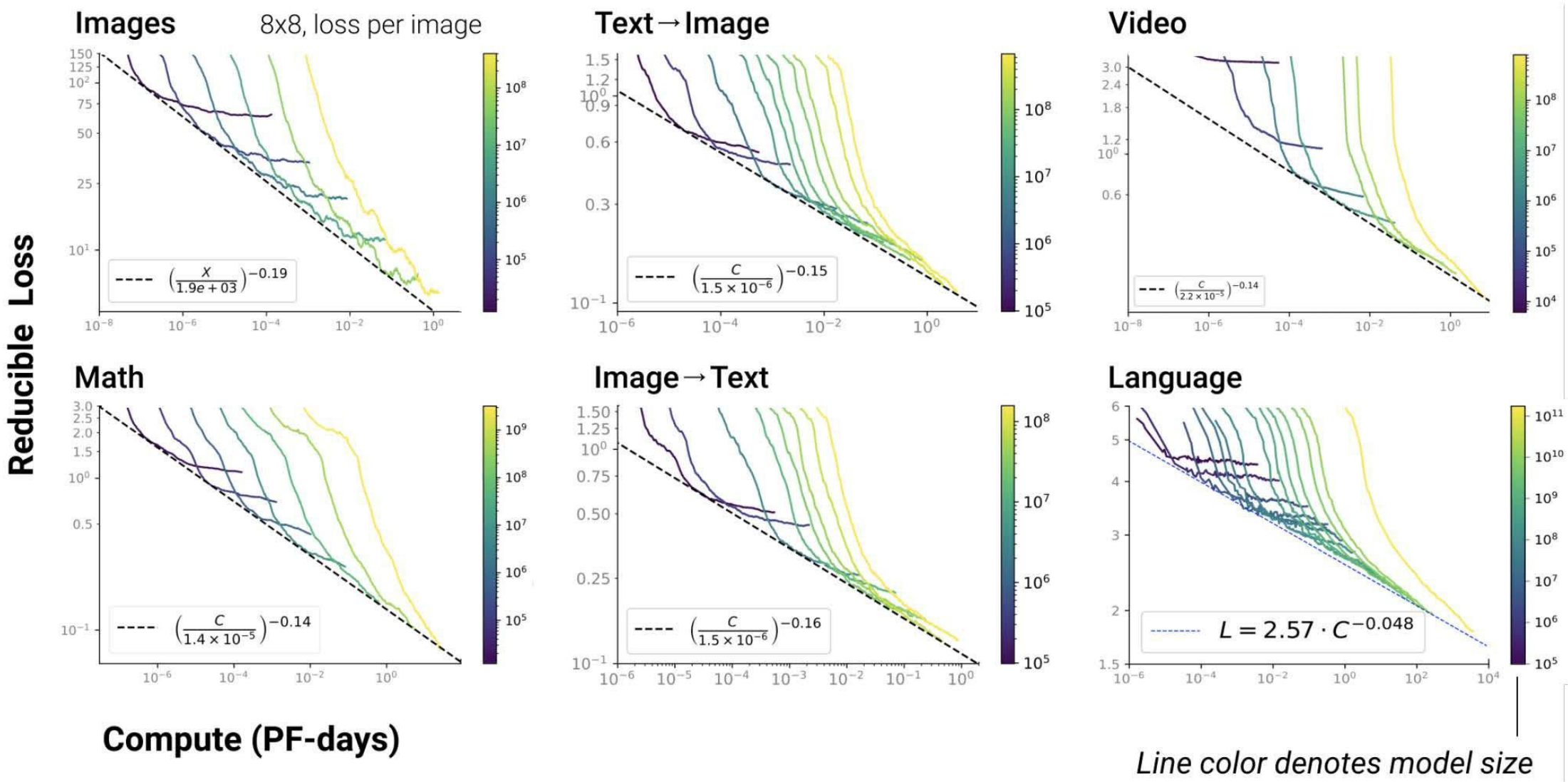
cost per block

projections and FFN grow with n; attention scores grow with n^2

Why n^2 matters for science

A 256 × 256 field with one token per grid point: n = 65,536, so each head of each layer computes $n^2 \approx 4.3 \times 10^9$ scores. With 16 × 16 patches, n = 256 and $n^2 \approx 6.6 \times 10^4$. Patching, local windows and linear-time approximations are the standard remedies.

Why the field converged on transformers: scaling



- Test loss falls as a smooth **power law** in compute — for images, video, math and language alike
- Compute-optimal training scales parameters and data together: roughly **20 tokens per parameter**

Kaplan et al. (2020) Scaling laws for neural language models · Henighan et al. (2020) Scaling laws for autoregressive generative modeling · Hoffmann et al. (2022) Training compute-optimal LLMs.

Strengths and limitations

Strengths

- Parallel over positions — fast to train
- Long-range interactions in one step; no vanishing gradients through time
- Scales predictably with data and compute
- Modality-agnostic: text, images, point clouds, fields
- Content-dependent mixing

Limitations

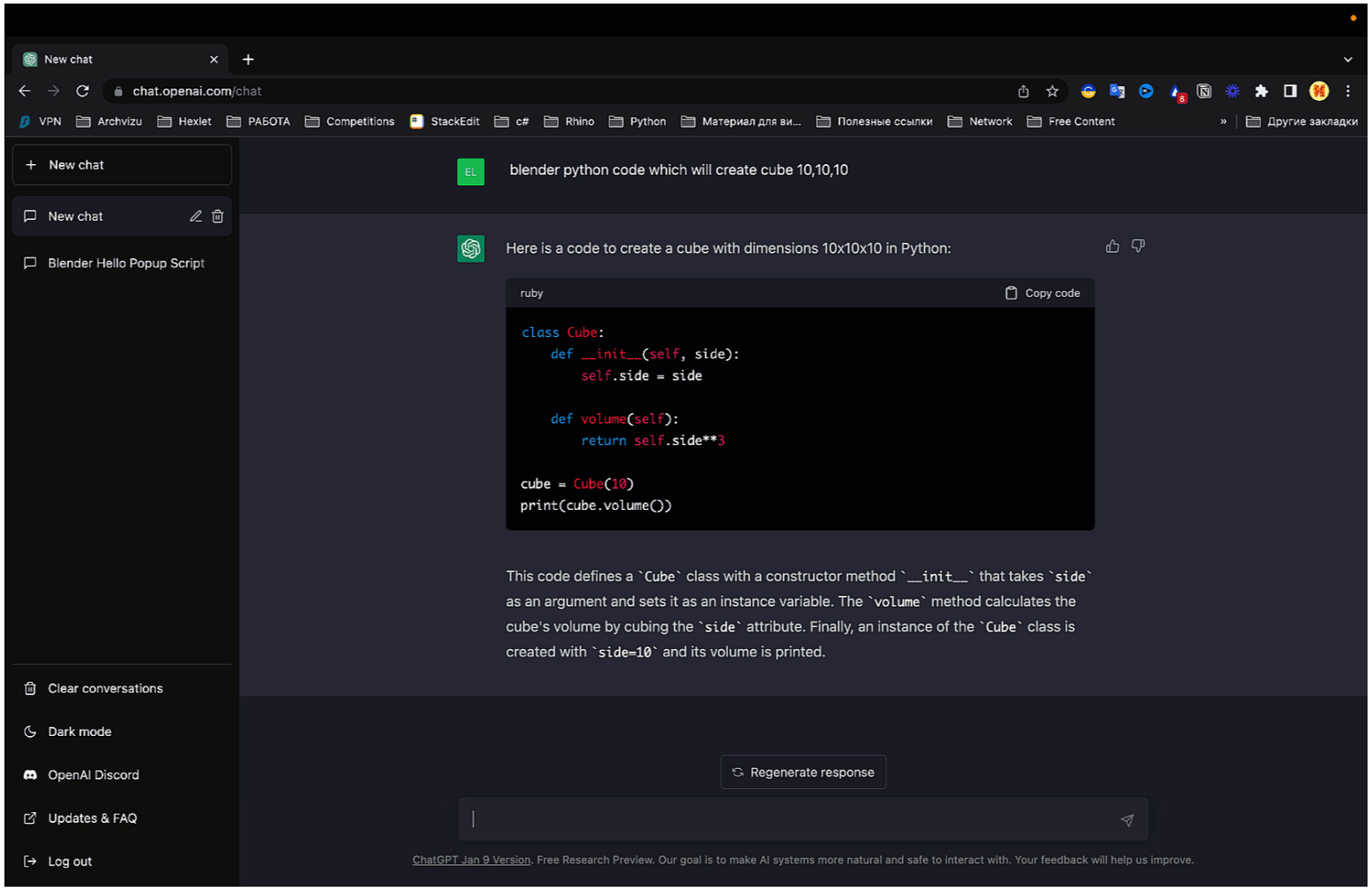
- Cost and memory grow as n^2 in sequence length
- Data hungry: weak inductive bias
- Expensive and delicate to train at scale
- No physics built in: symmetries, conservation, boundary conditions must be added
- Generation is still sequential

Deep learning highlights

What attention, scale and pretraining have made possible



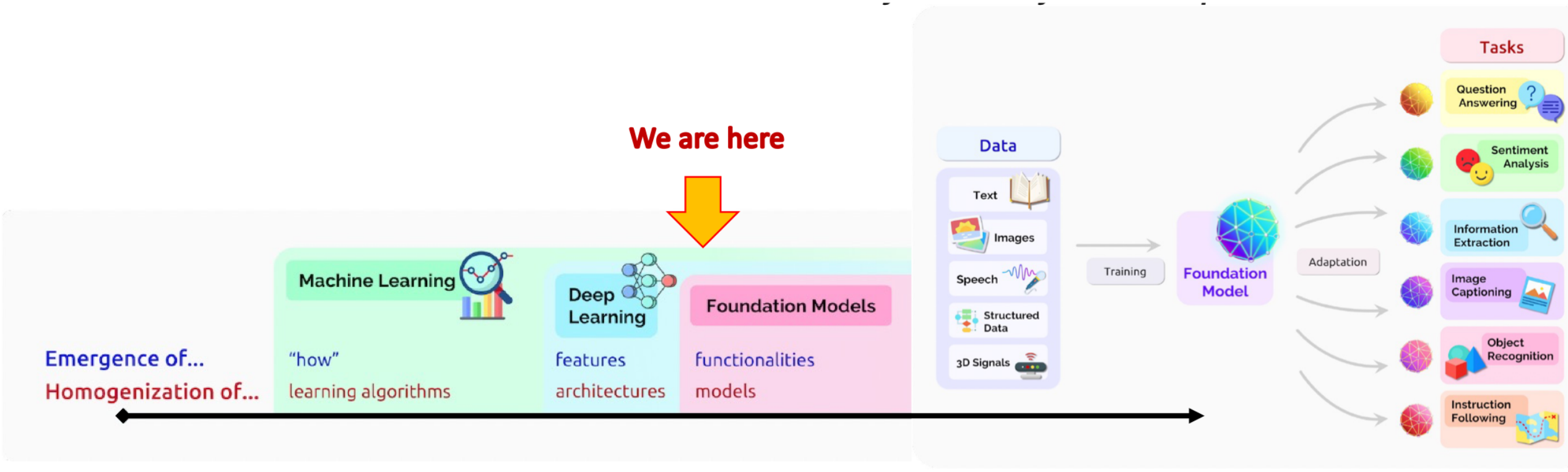
Generative pretrained transformers



- **Decoder-only** transformer, trained to predict the next token on a very large corpus
- Then adapted with instruction data and human feedback
- One model, many tasks: writing, coding, reasoning, tool use
- The same recipe — tokens, causal attention, maximum likelihood — that we wrote in 20 lines

Foundation Models

AI is undergoing a **paradigm shift** with the rise of models trained on broad data that can be adapted to a wide range of downstream tasks. We call these models **foundation models** to underscore their critically central yet incomplete character.



Emergence means that the behavior of a system is implicitly induced rather than explicitly constructed; it is both the source of scientific excitement and anxiety about unanticipated consequences.

Homogenization indicates the consolidation of methodologies for building AI systems across a wide range of applications; it provides strong leverage towards many tasks but also creates single points of failure.

Transfer Learning & Scale

On a technical level, foundation models are enabled by transfer learning and scale. Transfer learning is what makes foundation models possible, but scale is what makes them powerful.

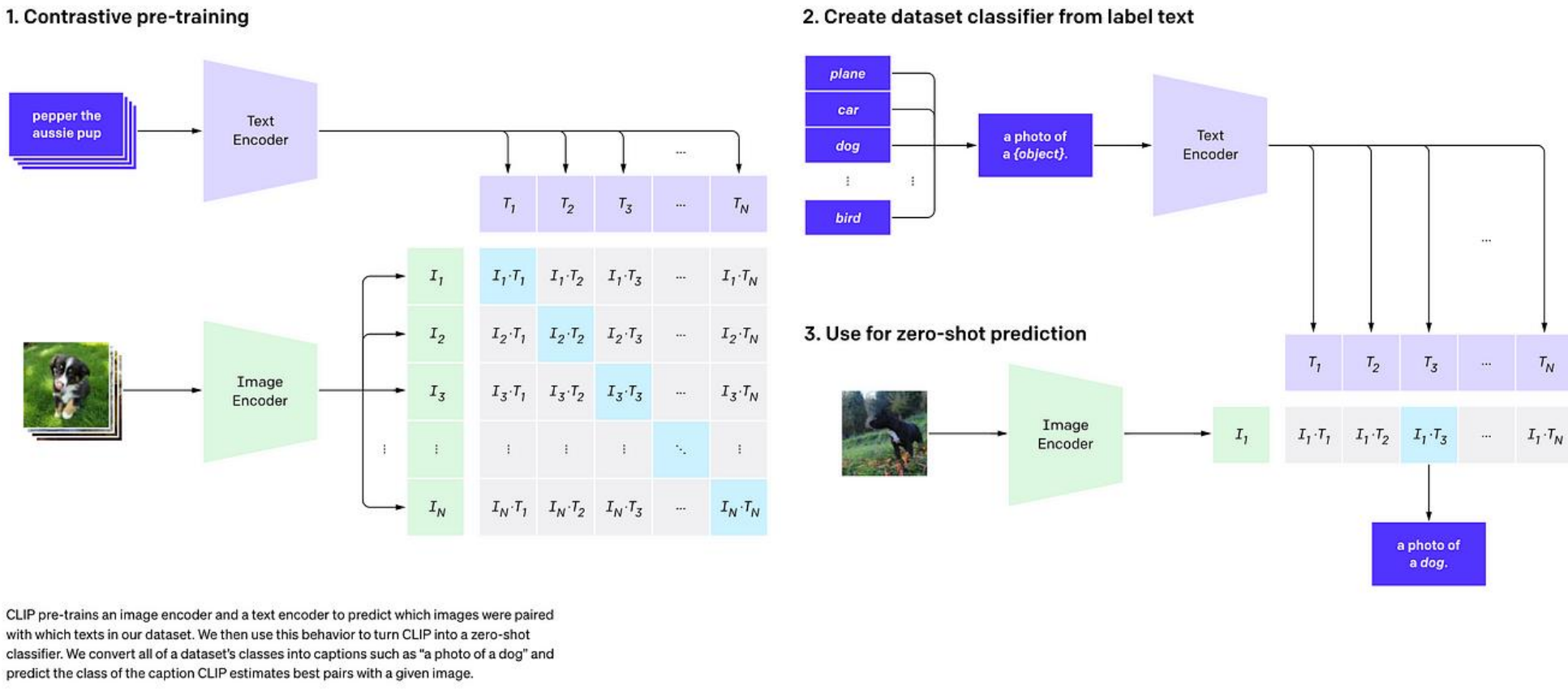
Scale required three ingredients:

i. improvements in **computer hardware** — e.g., GPU throughput and memory have increased 10× over the last four years;

ii. the development of the **Transformer** model architecture that leverages the parallelism of the hardware to train much more expressive models than before; and

iii. the availability of much more training **data**.

Vision–language models: CLIP



CLIP pre-trains an image encoder and a text encoder to predict which images were paired with which texts in our dataset. We then use this behavior to turn CLIP into a zero-shot classifier. We convert all of a dataset's classes into captions such as "a photo of a dog" and predict the class of the caption CLIP estimates best pairs with a given image.

Training: an image encoder and a text encoder map into one shared space. Matching image–caption pairs are pulled together, the other n^2 – n pairs pushed apart: a symmetric cross-entropy over the n × n similarity matrix.

Zero-shot: embed "a photo of a {class}" for every class name and pick the caption most similar to the image — a classifier written in words.

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t             - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

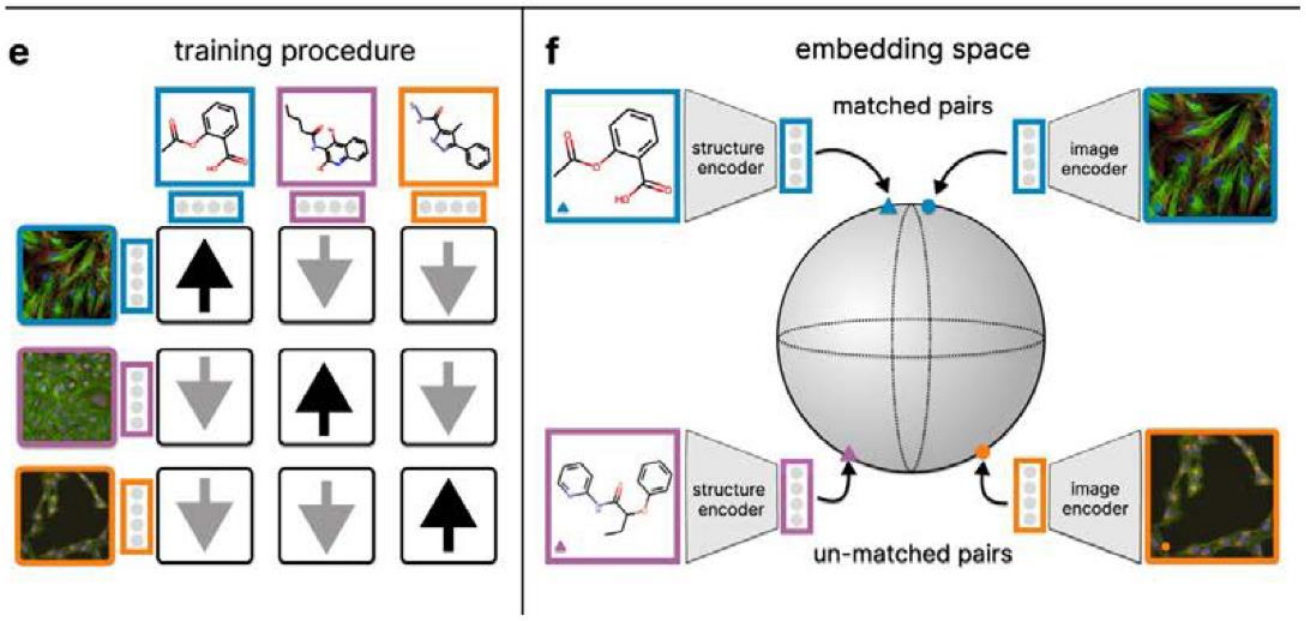
# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

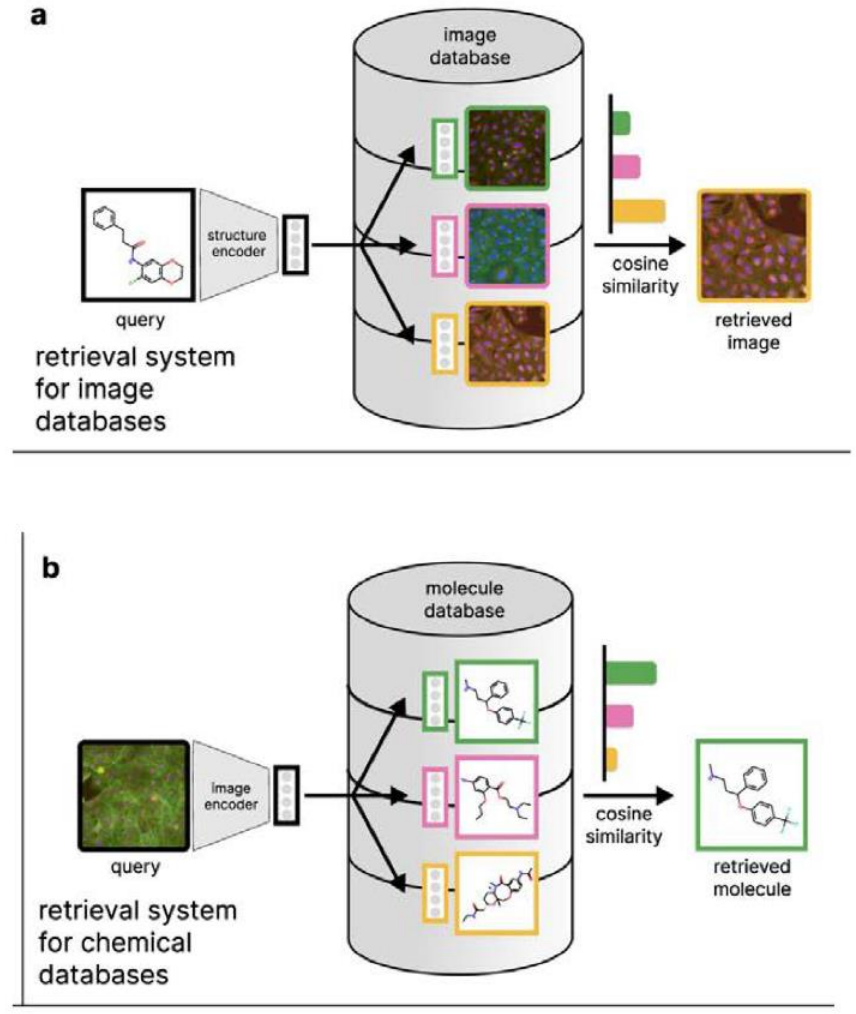

Radford et al. (2021). Learning transferable visual models from natural language supervision. ICML.

Querying bioimages with chemical structures

Dual-encoder for bioimage and molecular structure

Demonstrates transferability to various further prediction tasks within the domain of **drug discovery**

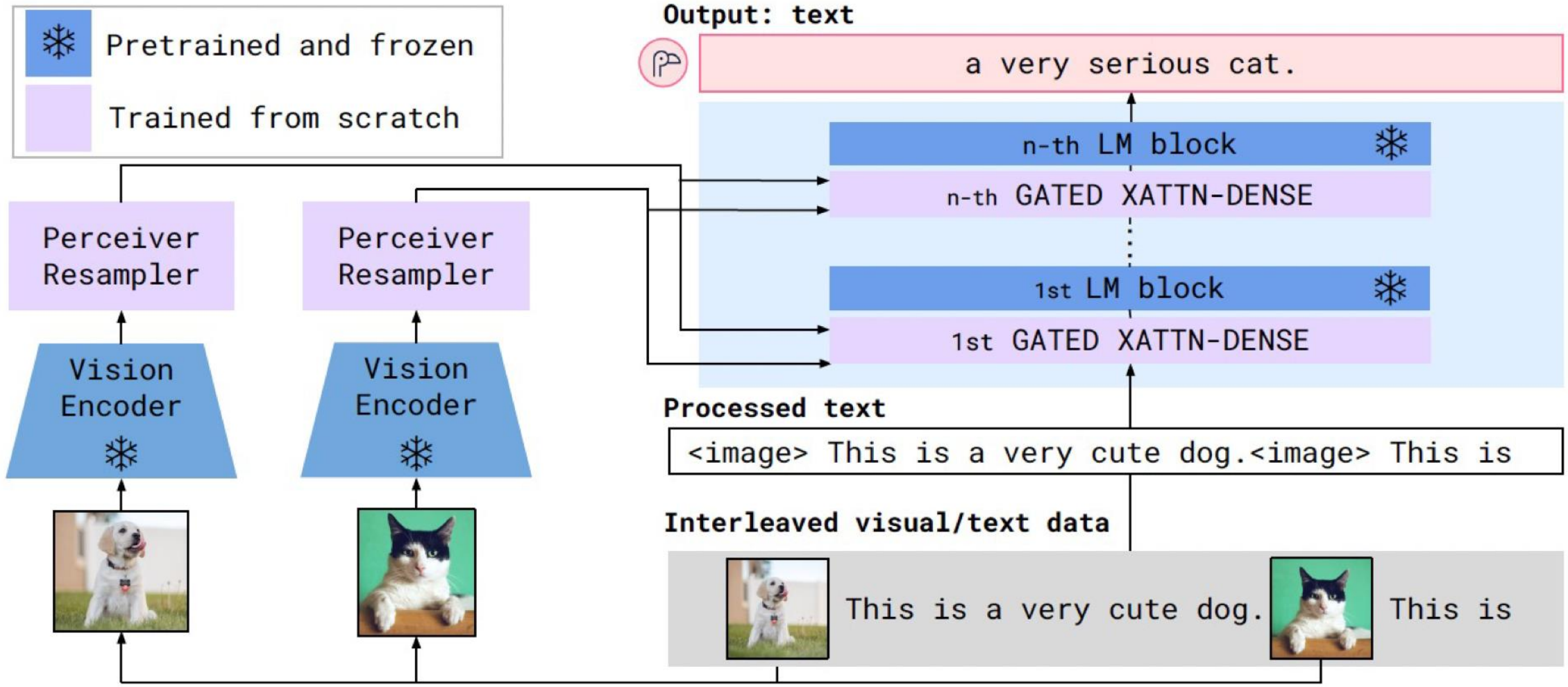




Ana Sanchez-Fernandez et al. Nature Comm. 2023

Extending LLMs with new modalities

Better prepares frozen vision encoder output for LLM processing.



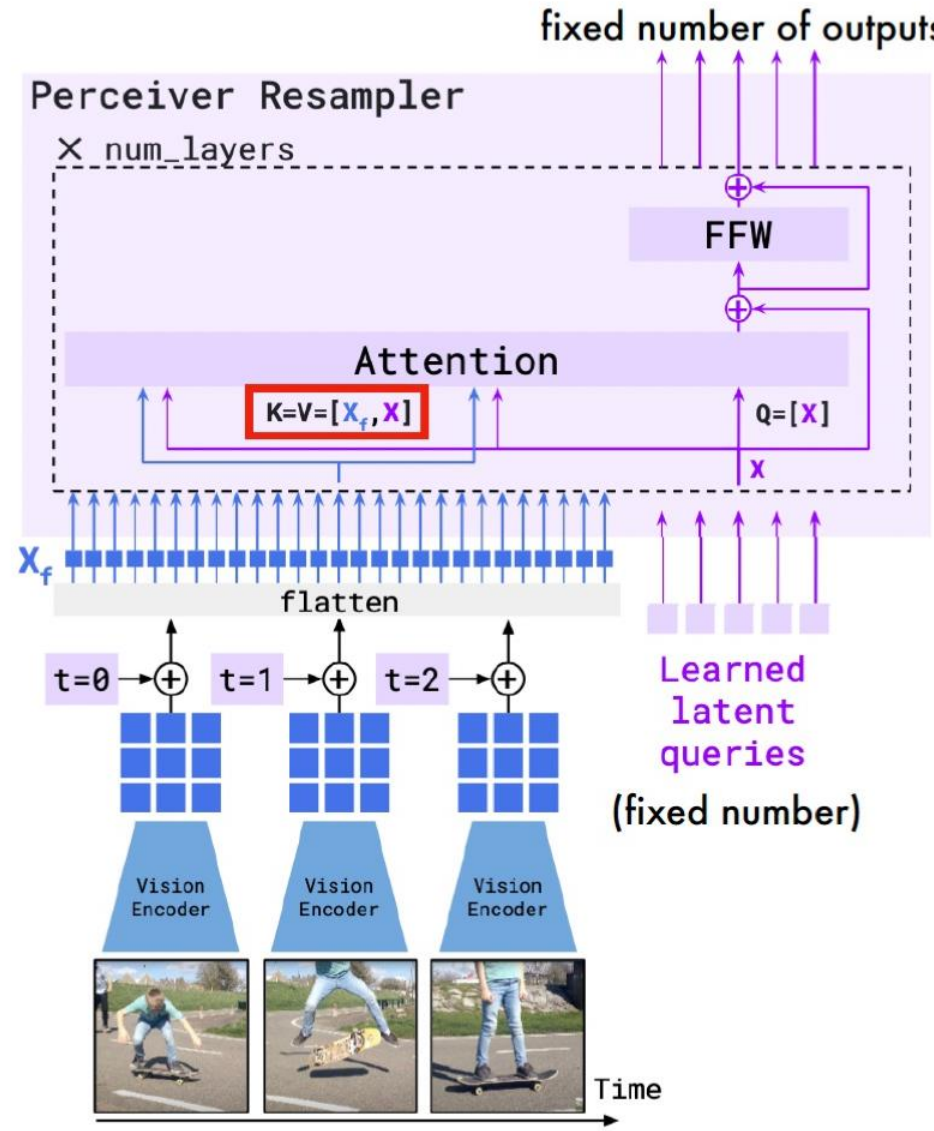
JB Alayrac et al. NeurIPS 2022

Perceiver Resampler

Connects the vision encoder to the frozen language model.

Outputs a fixed number of 64 visual tokens independent of varying input, from video or spatial resolution

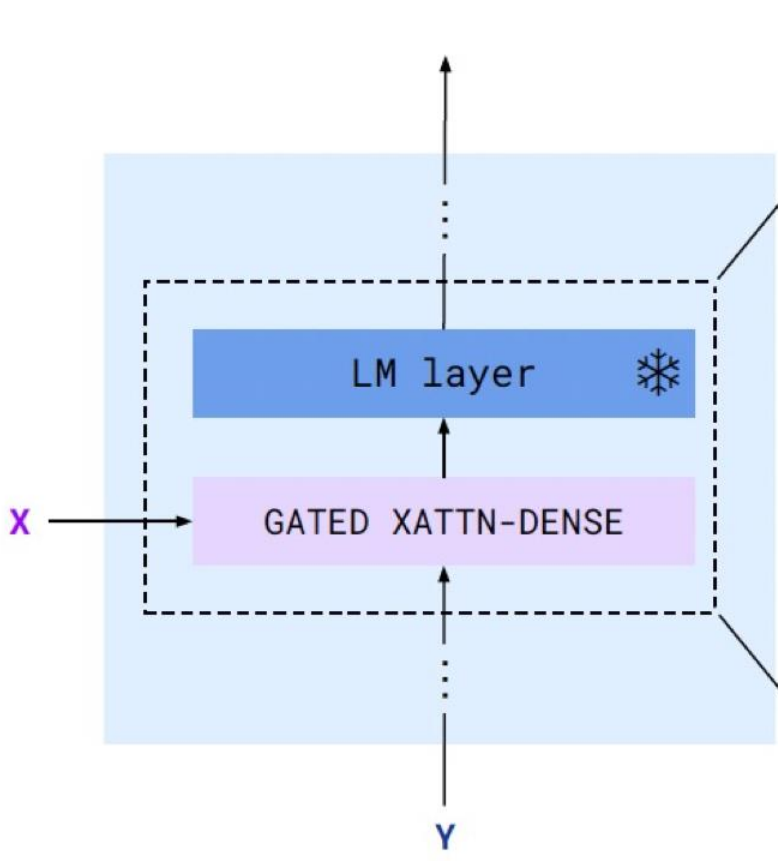
Reduces the computational complexity of the vision-text cross-attention.

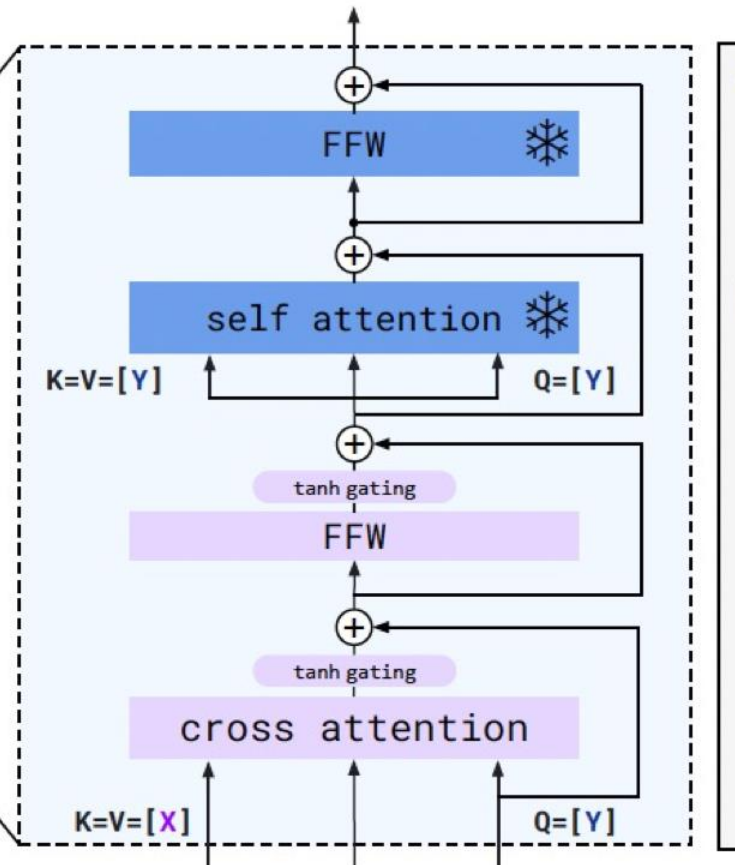


JB Alayrac et al. NeurIPS 2022

Gated X-attention

Freezes the pretrained LM blocks, and inserts **gated cross-attention dense** blocks between the original LM layers, that are trained from scratch.







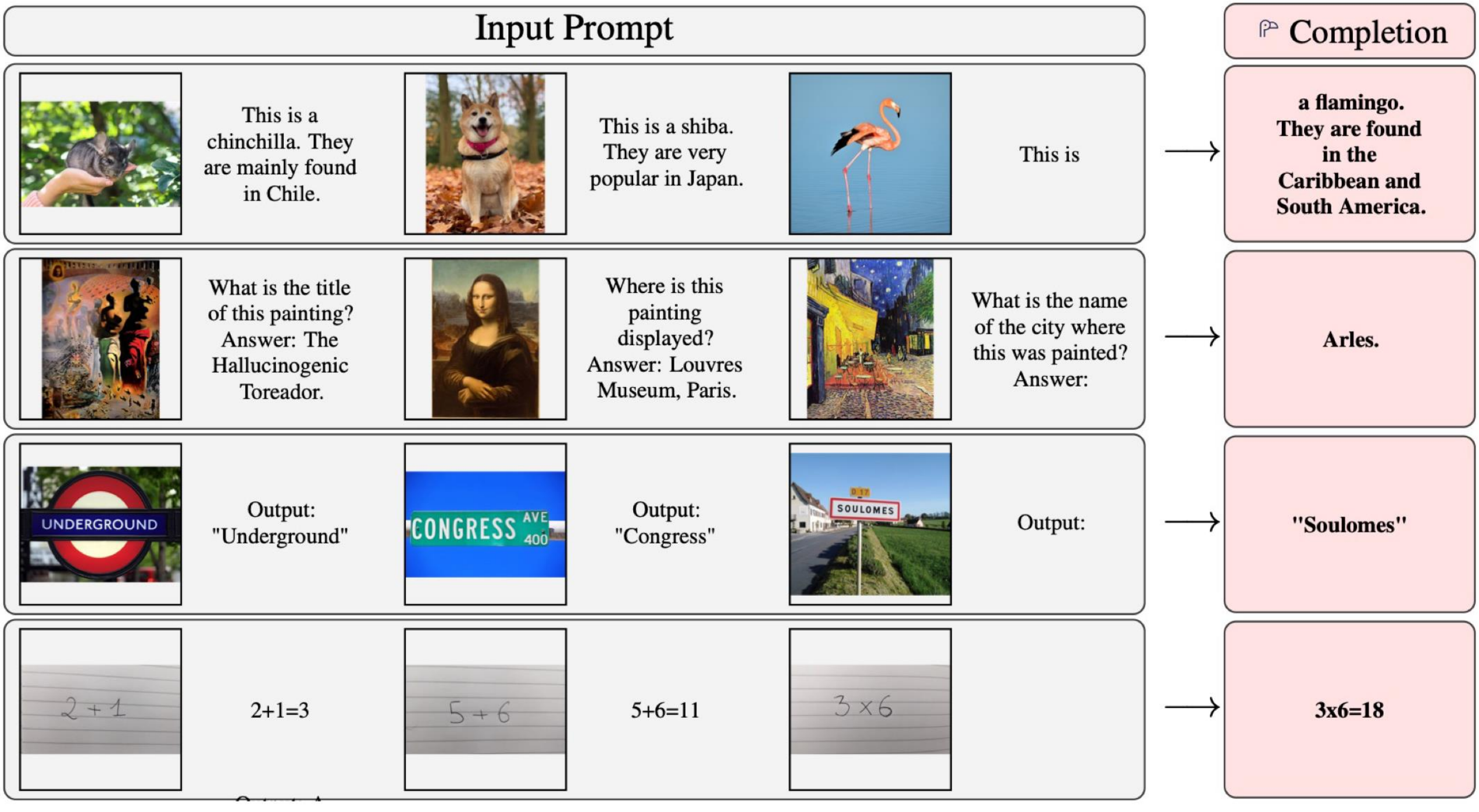

```
def gated_xattn_dense(
    y,  # input language features
    x,  # input visual features
    alpha_xattn, # xattn gating parameter – init at 0.
    alpha_dense, # ffw gating parameter – init at 0.
):
  """Applies a GATED XATTN-DENSE layer."""

  # 1. Gated Cross Attention
  y = y + tanh(alpha_xattn) * attention(q=y, kv=x)
  # 2. Gated Feed Forward (dense) Layer
  y = y + tanh(alpha_dense) * ffw(y)

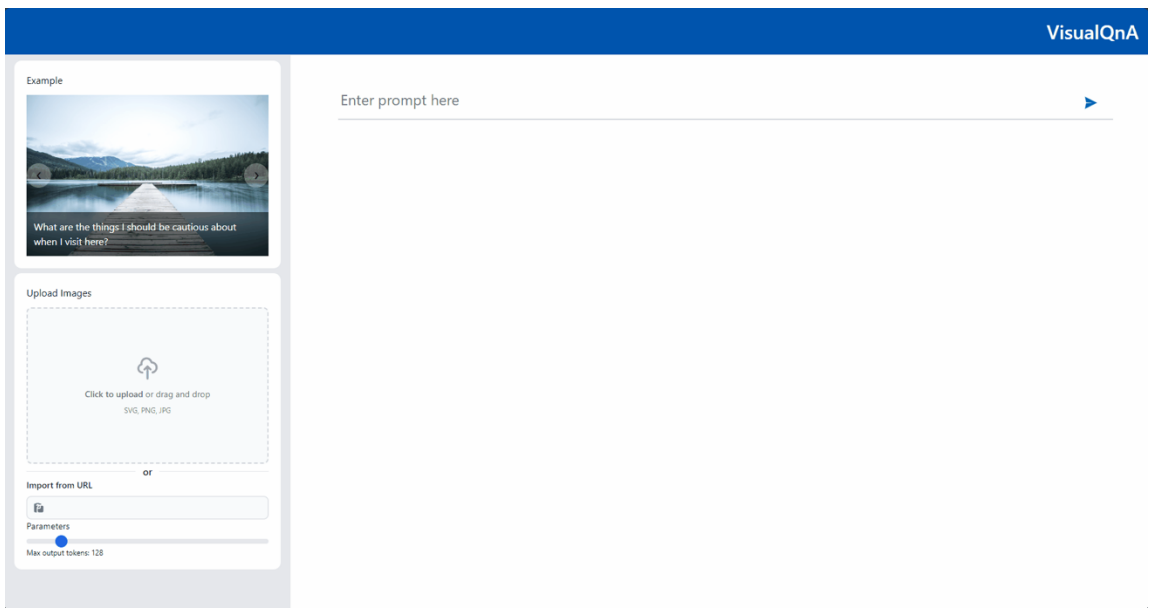
  # Regular self-attention + FFW on language
  y = y + frozen_attention(q=y, kv=y)
  y = y + frozen_ffw(y)
  return y  # output visually informed language features
```

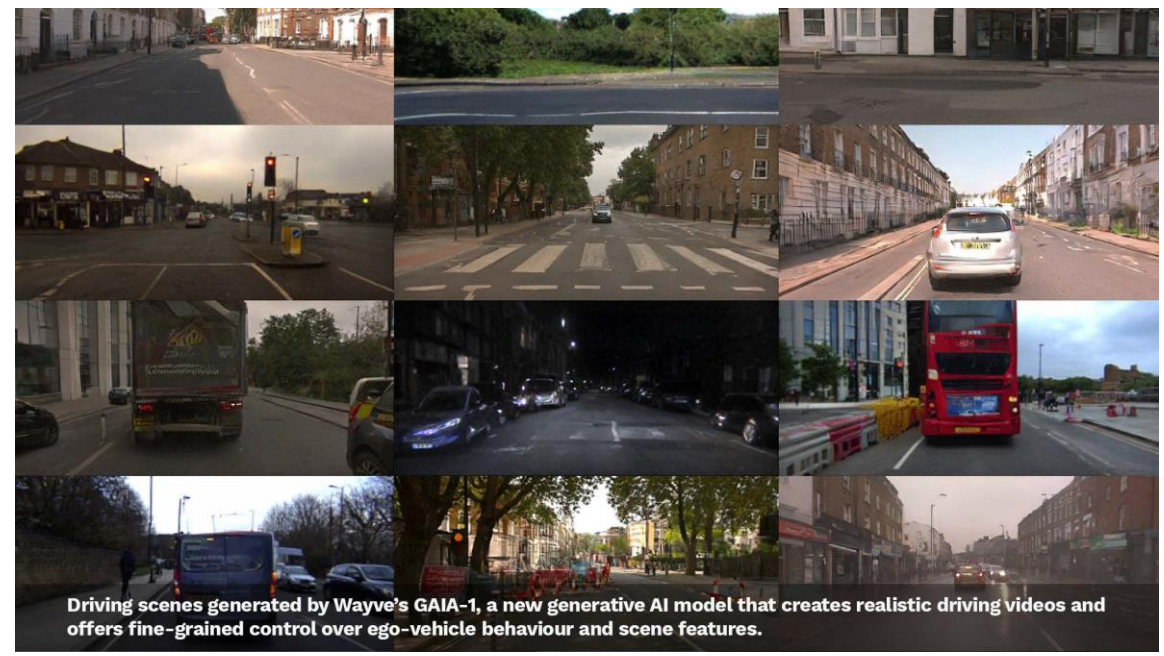
JB Alayrac et al. NeurIPS 2022

Few-shot Learning



Multimodal transformers: seeing, and imagining



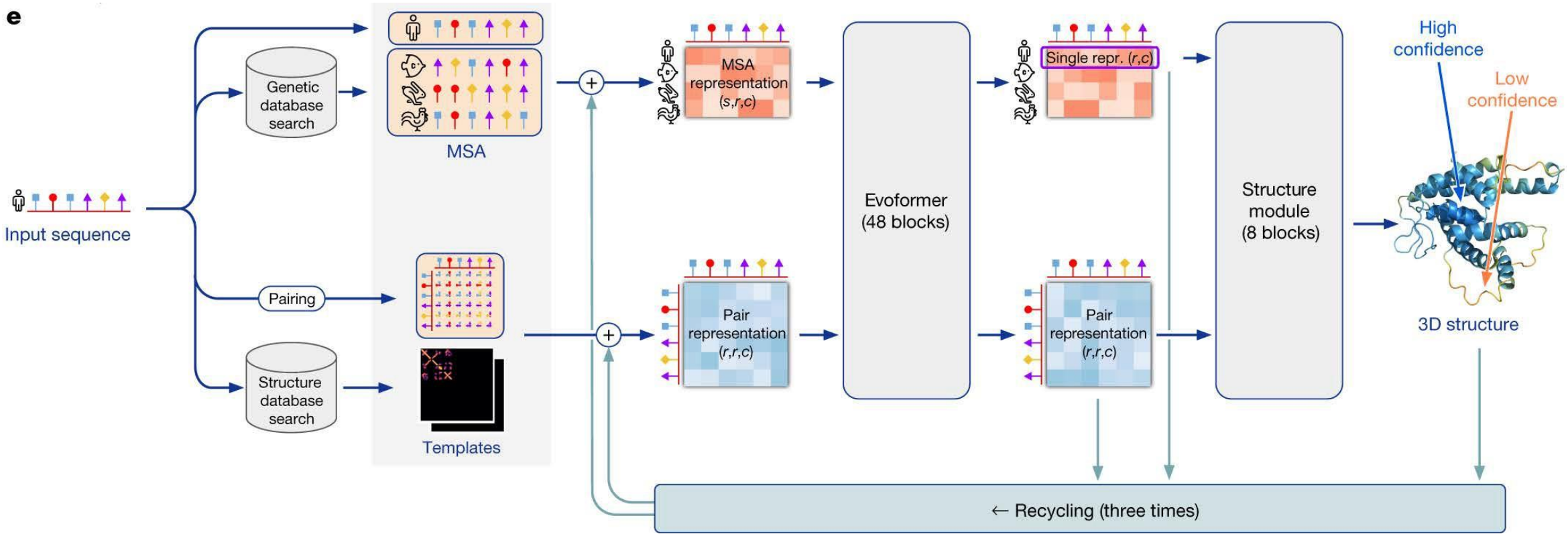


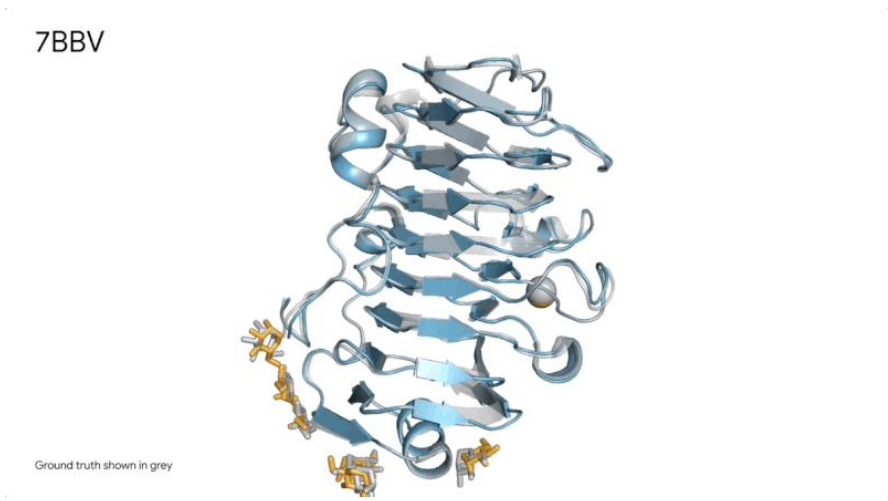
Vision-language assistants: image tokens and text tokens share one transformer, so a model can answer questions about a picture.

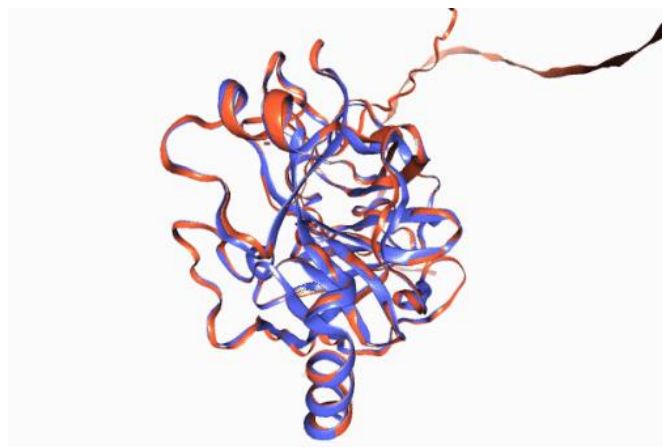
World models: GAIA-1 generates realistic driving video conditioned on text and actions — next-token prediction over video tokens.

ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices · *wayve.ai/thinking/introducing-gaia1*

AlphaFold: attention for protein structure



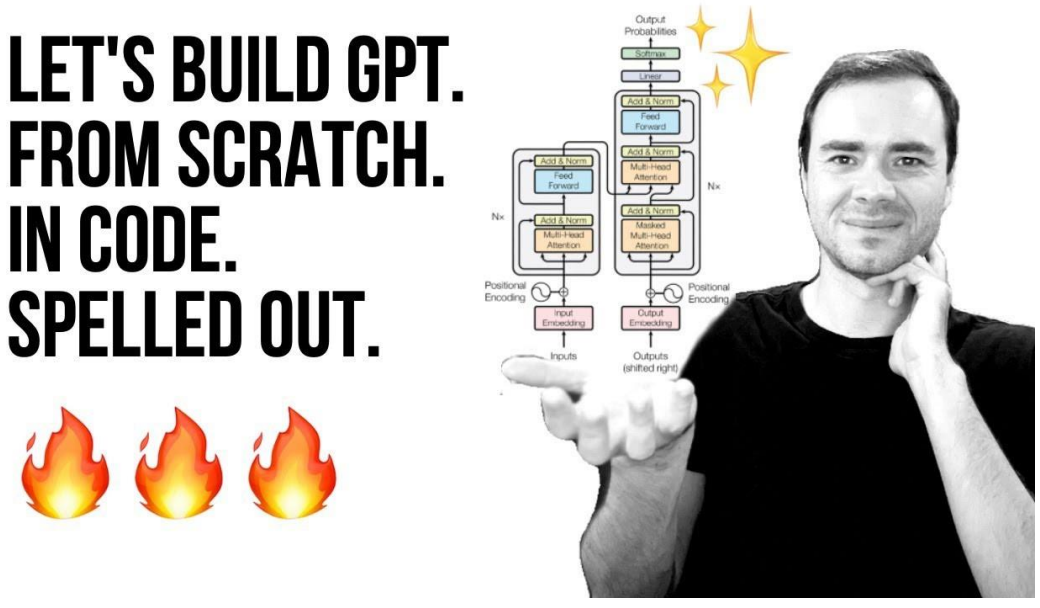




Attention over residues and over pairs of residues, refined by recycling. AlphaFold 3 extends this to complexes, with a diffusion module generating 3D coordinates.

Pipeline shown: Jumper et al. (2021), Nature (AlphaFold 2). Abramson et al. (2024), Nature (AlphaFold 3).

Build it yourself



- nanoGPT — a small, readable GPT in PyTorch

github.com/karpathy/nanoGPT

- "Let's build GPT: from scratch, in code, spelled out" (Karpathy, video)

www.youtube.com/watch?v=kCc8FmEb1nY

- UvA Deep Learning tutorials: Transformers and Vision Transformers

uvadlc-notebooks.readthedocs.io

- The Illustrated Transformer (Alammar)

jalammar.github.io/illustrated-transformer/

Course materials: Lecture 16 notes (attention) and Lecture 17 notes (the transformer), with the full pseudocode and Code Checkpoints 16–17.

- Formal Algorithms for Transformers (Phuong & Hutter)

arxiv.org/abs/2207.09238